

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

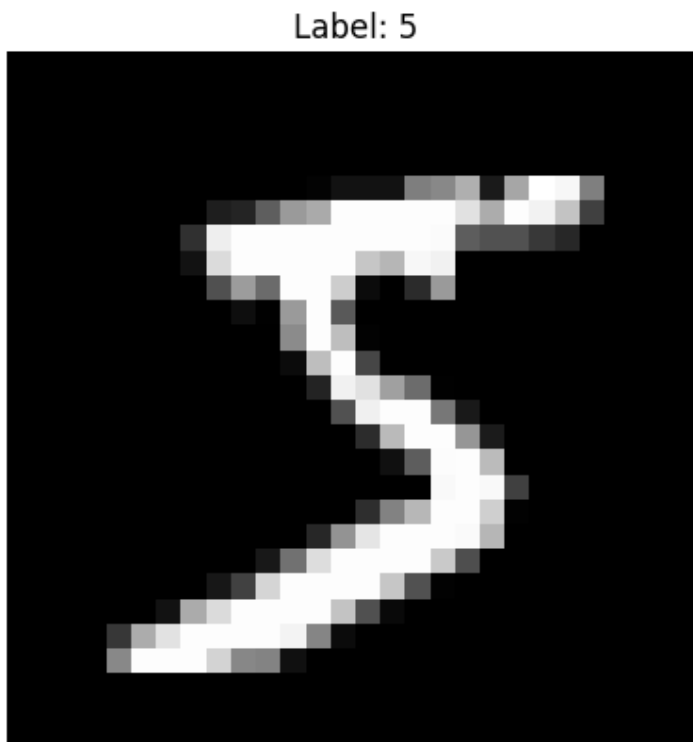
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.utils import to_categorical
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training Images Shape:", x_train.shape)
print("Training Labels Shape:", y_train.shape)
print("Testing Images Shape:", x_test.shape)
print("Testing Labels Shape:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist11490434/11490434 ————— 0s 0us/step
Training Images Shape: (60000, 28, 28)
Training Labels Shape: (60000,)
Testing Images Shape: (10000, 28, 28)
Testing Labels Shape: (10000,)
```

```
plt.imshow(x_train[0], cmap='gray')
plt.title("Label: {}".format(y_train[0]))
plt.axis("off")
plt.show()
```



```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

```
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resizing/flatten.py:37: U
super().__init__(**kwargs)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None , 784)	0
dense (Dense)	(None , 128)	100,480
dense_1 (Dense)	(None , 64)	8,256
dense_2 (Dense)	(None , 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

```
Epoch 1/10
1500/1500 ————— 7s 4ms/step - accuracy: 0.9215 - loss: 0.2731 - val_
Epoch 2/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9660 - loss: 0.1141 - val_
Epoch 3/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9764 - loss: 0.0777 - val_
Epoch 4/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9809 - loss: 0.0592 - val_
Epoch 5/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9851 - loss: 0.0467 - val_
```

```
Epoch 6/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9877 - loss: 0.0372 - val_
Epoch 7/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9897 - loss: 0.0300 - val_
Epoch 8/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9916 - loss: 0.0258 - val_
Epoch 9/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9921 - loss: 0.0227 - val_
Epoch 10/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9941 - loss: 0.0172 - val_
```

```
loss, accuracy = model.evaluate(x_test, y_test)
```

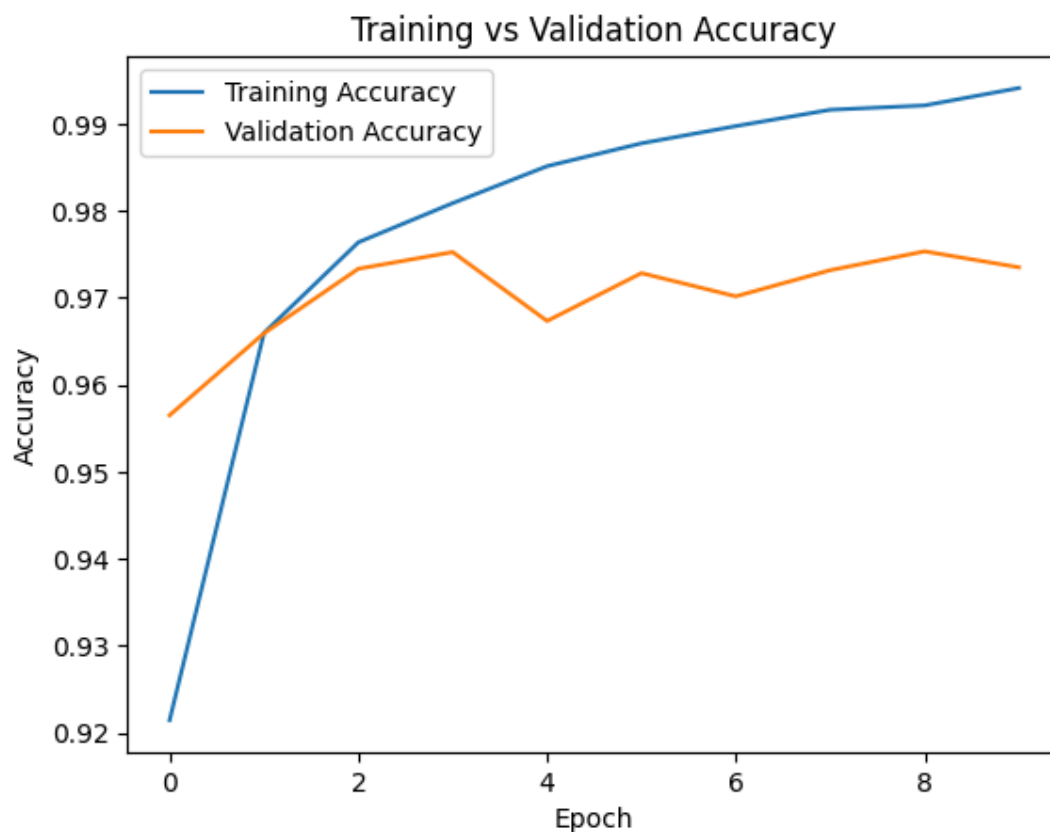
```
print("\nTest Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.9749 - loss: 0.1023
```

```
Test Loss: 0.10228846222162247
```

```
Test Accuracy: 0.9749000072479248
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```



```
prediction = model.predict(x_test)
```

```
predicted_digit = np.argmax(prediction[0])
```

```
predicted_digit = np.argmax(predicted_logits)\nactual_digit = np.argmax(y_test[0])\n\nprint("Predicted Digit:", predicted_digit)\nprint("Actual Digit:", actual_digit)
```

313/313 ————— 1s 1ms/step

Predicted Digit: 7

Actual Digit: 7

```
plt.imshow(x_test[0], cmap='gray')\nplt.title("Predicted: {}".format(predicted_digit))\nplt.axis("off")\nplt.show()
```

Predicted: 7

